PARADOX // REFERENCE

Introduction

Architecture Overview

End-to-End Data Flow

Layer 1: Configuration

Layer 2: Ollama Client

Layer 3: Vector Memory

Layer 4: Symbolic Router

Layer 5: Agent Loop

Layer 6: System Prompts

Layer 7: Answer Formatting

Tool Catalog

- Web Search
- Deep Research
- Fetch URL
- Python Analysis
- EEG Pipeline
- Memory Tools
- System Tools

EEG Analysis Deep Dive

- 24-Step Pipeline
- Signal Processing Math
- File Formats
- Software Ecosystem
- Interactive Outputs & Jobs
- Bundled EEG Worker
- Clinical & NetOps Layers
- Phenotype-Informed Analysis

Advanced Visualizations

Reflection Engine

GPU & Memory Optimization

Flash Attention

KV Cache Quantization

Embedding Quantization

TurboQuant Theory

Desktop Application

Tauri Shell

Sidecar & Worker

Build Pipeline

First-Run Setup

Licensing System

Cryptographic Design

Machine Binding

Feature Flags

File Management

Version Tracking

Frontend & HUD

REST API

Use Cases

Paradox Solutions LLM

A Technical Reference for the Curious, the Skeptical, and the Slightly Obsessed — v0.3.1

The universe, as everyone who has ever tried to parse a malformed JSON response knows, is fundamentally hostile to good data. It is a truth universally acknowledged that a local language model in possession of a good GPU must be in want of a proper architecture around it. Otherwise it just sits there, hallucinating URLs and insisting that the capital of France is "approximately Paris, confidence 0.4."

This document describes **Paradox Solutions LLM**: a fully local AI research assistant that runs on your hardware, sends nothing to the cloud, and has been taught — through patient engineering, not magic — to do useful work with EEG data, web research, and the kind of multi-step reasoning that makes smaller models sweat. As of **v0.3.1**, the science stack ships *inside the installer*: a dedicated **EEG worker executable** carries MNE, SciPy, Plotly, and everything else the visualization pipeline needs, so that the operator need not wrestle a Python venv into shape before the brain can speak. The most dangerous phrase in neurotech — "we've always required a separate science install" — no longer applies.

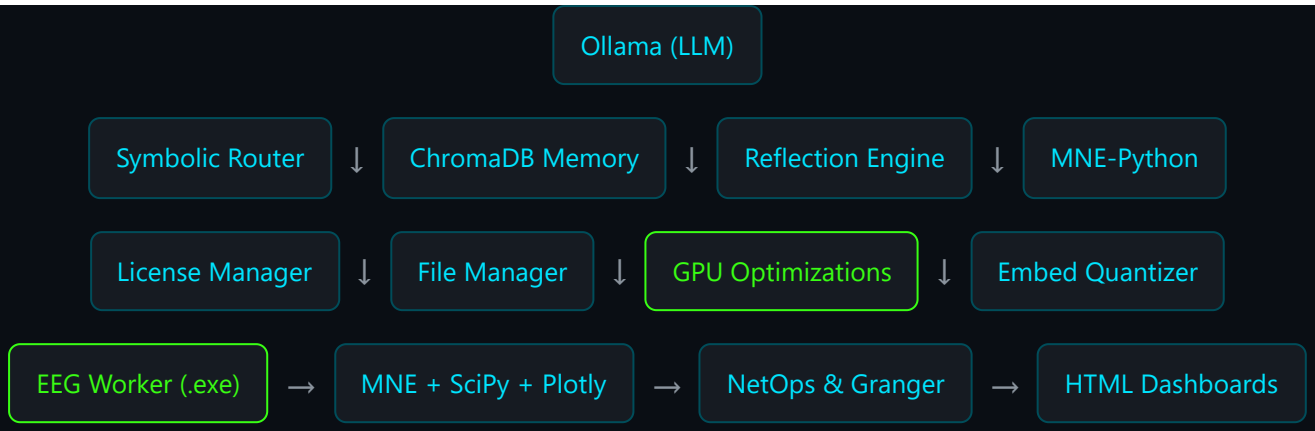
Every layer is documented here. Every formula. Every decision about why the system works the way it does, rather than some other way that would have been easier to build and harder to trust. A ship in harbor is safe, but that is not what ships are built for — and this system was built to sail.

> *A system that cannot explain itself is not an assistant. It is a liability with a chat interface.*

Architecture Overview

Paradox is seven layers of software wrapped around two local language models, a vector database, and — as of v0.3.1 — a frozen science worker that carries an entire EEG analysis laboratory in a single executable. Each layer does exactly one job. None of them are optional in the way that seatbelts are optional — technically you can remove them, but the first time you hit a wall you'll wish you hadn't. Million-to-one chances of getting away with it crop up nine times out of ten, but only in fiction.

Tauri Desktop → Next.js UI → FastAPI → Agent Loop → Tool Registry →



Layer	Technology	Responsibility
Desktop	Tauri v2 (Rust)	Native window, system tray, sidecar management, NSIS/MSI installers
Frontend	Next.js, React, Tailwind	Chat UI, file upload, license panel, optimization status
API	FastAPI, Pydantic	HTTP endpoints, CORS, file management, license proxy
Agent	Python (ReAct v2)	Think → Act → Observe → Answer loop
Symbolic	YAML + regex	Deterministic tool routing hints
LLM	Ollama (Qwen3:8B, phi4-mini)	Reasoning, generation, classification (flash attention + quantized KV cache)
Memory	ChromaDB + nomic-embed-text	Persistent vector storage with weighted scoring + quantized embeddings
Optimization	Flash Attn, KV quant, embed quant	VRAM efficiency, inference speed, storage compression
EEG Worker	PyInstaller (frozen)	Bundled `paradox-eeg-worker.exe`: MNE, SciPy, Matplotlib, Plotly, scikit-learn, networkx, statsmodels — runs viz subprocesses without a separate Python install
Licensing	RSA, machine fingerprint	Offline activation, feature flags, per-machine binding
Tools	Python modules	Web search, EEG analysis, research, system info (license-gated)

End-to-End Data Flow

When you type a question and press Enter, here is what actually happens. Not approximately. Not "the LLM figures it out." *Exactly* this:

1. The **Next.js frontend** sends `POST /agent` with your message to port 8765.
2. FastAPI validates the request and calls `run_agent()`.
3. The **dual-model router** asks phi4-mini: "Is this DIRECT or AGENT?" If DIRECT (simple factual question), phi4-mini answers immediately — no tool calls, no agent loop. Response in ~2 seconds.
4. If AGENT, the full **ReAct loop** begins. The system prompt is assembled from: static rules, current date, EEG domain knowledge (if enabled), tool descriptions, the current focus document, and any neuro-symbolic hints triggered by your query.
5. Qwen3:8B generates a JSON response: `{"thought": "...", "action": "web_search", "action_input": {"query": "..."}, "answer": null, "confidence": 0.0}`
6. The **tool registry** dispatches the action to the appropriate tool module.
7. The tool runs, produces a `ToolResult`, and the observation is formatted and injected back into the conversation.
8. The model reads the observation and either calls another tool or produces a final answer.
9. The answer is passed through **URL validation** (fabricated links are stripped) and **Markdown formatting**.
10. The conversation is embedded and stored in **ChromaDB** for future recall.
11. The formatted answer returns to the frontend and renders as rich Markdown with clickable links.

Maximum agent steps: **8**. In practice, most queries resolve in 2. The system is designed to be parsimonious with compute — every tool call costs time, and time on a local GPU is not free.

Layer 1: Configuration

All configuration flows from a single Pydantic `Settings` class that reads `.env` at startup. Nothing is hardcoded that should be soft. Nothing is soft that should be hard. The distinction between the two is the thing that separates engineering from typing.

▼ **Full Configuration Reference**

Setting	Default	Purpose

`chat_model`	`qwen3:8b`	Primary reasoning model
`fast_model`	`""` (disabled)	Lightweight router/answerer (e.g. `phi4-mini`)
`embed_model`	`nomic-embed-text`	Embedding model for ChromaDB vectors
`ollama_chat_num_ctx`	`16384`	Context window size (tokens)
`ollama_chat_num_predict`	`4096`	Max generation tokens per turn
`agent_max_steps`	`8`	ReAct loop budget
`confidence_threshold`	`0.75`	Minimum confidence to accept an answer
`memory_w_sim`	`0.45`	Weight: cosine similarity
`memory_w_imp`	`0.20`	Weight: importance score
`memory_w_rec`	`0.20`	Weight: recency decay
`memory_w_boost`	`0.15`	Weight: manual boost
`memory_recency_half_life_days`	`14.0`	Exponential decay half-life
`eeg_domain_enabled`	`true`	Inject EEG knowledge into system prompt
`python_analysis_timeout`	`300`	Max seconds for MNE scripts
`symbolic_enabled`	`true`	Enable regex-based tool routing
`web_search_auto_fetch`	`true`	Auto-fetch and parse search result pages
EEG subprocess resolution (v0.3.1)		
`PARADOX_EEG_PYTHON`	*(env var)*	Override the interpreter used for EEG visualization subprocesses. When set, the API uses this path instead of auto-detection. Preferred name.
`EEG_PYTHON`	*(env var)*	Legacy alias for `PARADOX_EEG_PYTHON`. Checked when the preferred name is absent.

When the API is running as a frozen PyInstaller bundle, it prefers **`paradox-eeg-worker.exe`** from the `eeg-worker/` directory beside the main binary. If the worker is absent, it falls back to the environment-variable override, then to `py -3`, then to `python` on `PATH`. One accurate measurement — confirmed by `GET /health` →

`eeg_subprocess` — is worth a thousand expert opinions about which interpreter the system might be using.

Layer 2: Ollama Client

The Ollama client is the system's mouth and ears. It speaks to the local Ollama server over HTTP, and its primary job is to make the models' output *parseable* — which, with local models, is about as straightforward as herding cats through a JSON schema.

Light thinks it travels faster than anything, but a read timeout finds the darkness has always got there first. When Ollama takes longer than the configured `httpx` read timeout to generate a response — a large model, an overloaded GPU, the digital equivalent of an electrode with too much impedance — the system now returns a clean **504 Gateway Timeout** with a human-readable message, rather than an unhandled 500 that tells the operator nothing useful. If you can cut a nanosecond of wire and hold it in your hand, you can certainly hold a timeout in a proper exception handler.

Key Operations

- `chat(model, messages, temperature, format_json)` — Sends messages to `/api/chat`. When `format_json=True`, sets Ollama's format parameter to enforce JSON output.
- `embed(text)` — Calls `/api/embeddings` with `nomic-embed-text` to produce 768-dimensional vectors for ChromaDB storage.
- `strip_thinking(text)` — Qwen3 emits `<think> … </think>` reasoning blocks. This strips them before the output reaches the user. The thinking happens; it just doesn't clutter the response.
- `parse_json_loose(text)` — The real hero. Tries `json.loads` first. On failure, finds the first `{` and last `}` and tries again. This handles the majority of "almost valid JSON" that local models produce — trailing commas, preamble text, nested explanation before the actual object.

Timeout Handling (v0.3.1)

The `POST /agent` route catches `httpx.ReadTimeout`, `httpx.ConnectTimeout`, and `httpx.ConnectError` from the Ollama call and translates them into proper HTTP status

codes:

Exception	HTTP Status	Meaning
`ReadTimeout`	**504**	Ollama accepted the request but didn't finish in time
`ConnectTimeout`	**504**	Could not reach Ollama within the connect window
`ConnectError`	**502**	Ollama is not running or unreachable

Layer 3: Vector Memory

Memory is what separates a useful assistant from a goldfish with delusions of competence. Paradox stores everything in **ChromaDB** with three persistent collections:

Collection	Contents	Typical Entries
`chat_history`	Every conversation turn (user + assistant)	Thousands over weeks of use
`documents`	Ingested files, reflection outputs	Research papers, clinical notes, reflections
`library`	Chunked reference material	PDFs, EEG reference docs, protocol guides

The Scoring Formula

Raw ChromaDB returns a *distance*. Distance is not relevance. The system converts distance to similarity and then blends four signals into a single score:

$$\text{similarity} = 1 / (1 + \text{distance})$$

$$\text{recency} = e^{-\text{age_days} / \text{half_life}}$$

$$\text{score} = w_1 \cdot \text{similarity} + w_2 \cdot \text{importance} + w_3 \cdot \text{recency} + w_4 \cdot \text{manual_boost}$$

Where weights are **normalized** to sum to 1.0 before blending:

$$w_i = raw_w_i \,/\, \Sigma\, raw_w_j$$

▼ **Default weight breakdown**

Signal	Raw Weight	Normalized	What It Measures
Similarity	0.45	45%	Cosine distance from the query embedding
Importance	0.20	20%	Assigned at ingestion; bumped +0.02 on each access (capped at 1.0)
Recency	0.20	20%	Exponential decay: half-life of 14 days
Manual Boost	0.15	15%	Operator or tool can mark memories as important

The **recency decay** uses a natural exponential with configurable half-life. A memory from 14 days ago scores 0.5 on recency. A memory from 28 days ago scores 0.25. A memory from yesterday scores ~0.95. This means the system naturally forgets noise while preserving things you keep asking about (which get importance bumps on access).

The formula is deliberately simple. Complex scoring functions are difficult to debug, and when your memory system returns the wrong document, you need to be able to look at four numbers and say "ah, the recency killed it" — not stare at a neural attention matrix wondering which head made the mistake.

Layer 4: Neuro-Symbolic Router

Before the LLM even sees your message, a set of regex patterns fires against it. This is not AI. This is the computational equivalent of a well-organized filing cabinet — deterministic, fast, and immune to hallucination.

Each rule is a YAML entry with a regex `pattern` and a `text` hint. If the pattern matches, the hint is injected into the system prompt. The LLM reads it as guidance, not as a command — a suggestion from someone who has seen this kind of question before.

```yaml
# From default_rules.yaml
- pattern: "(?i)(QEEG|quantitative EEG|brain.?map|z.?score)"
  text: >
    QEEG involves computing absolute/relative band power per channel
    and comparing to normative databases. Use run_python_analysis
    with MNE for PSDs, then plot topomaps ...
```

There are **30+ rules** covering: general system queries, EEG software platforms (NeuroGuide, BrainMaster, BioExplorer, BioEra, Cygnet, OpenBCI, EEGLAB, Brain Products), analysis methods (spectral, ERP, ICA, connectivity, source localization), file formats, and research queries.

The rules engine has zero latency cost. A regex match takes microseconds. The hint text adds maybe 200 tokens to the prompt. The payoff is that an 8B model, which might otherwise flail when asked about sLORETA inverse solutions, gets a roadmap in its context window before it starts generating.

> *Deterministic routing layered on probabilistic inference. The filing cabinet tells the imagination where to look.*

Layer 5: The Agent Loop (ReAct v2)

The agent loop is where thinking happens. Not in the metaphysical sense — in the very literal sense of "the model outputs a JSON object with a field called `thought`."

The ReAct Pattern

Each step, the model produces:

```json
{
  "thought": "I need to find the 2026 Suisun Summit dates ... ",
  "action": "web_search",
  "action_input": {"query": "suisun neuroscience summit 2026"},
  "answer": null,
  "confidence": 0.0
}
```

When the model has enough information, it sets `action` to null and fills in the `answer`:

```
{
  "thought": "The search found the official page ... ",
  "action": null,
  "action_input": {},
  "answer": "The 2026 Suisun Neuroscience Summit ... ",
  "confidence": 0.85
}
```

Dual-Model Routing

The Fast Path (phi4-mini)

Simple questions — "what is the capital of France," "convert 5 miles to km" — don't need web search, tool calls, or multi-step reasoning. The router asks phi4-mini to classify the query as `DIRECT` or `AGENT`. Direct queries get answered immediately by phi4-mini itself. This saves 30-50 seconds of unnecessary agent loop time for questions the model already knows.

Safety Rails

- **Repeat tool guard:** If the model calls the same tool twice, the loop force-stops and extracts a fallback answer from whatever observations exist. Scientists have calculated that the chances of an 8B model getting stuck in a loop are significant; this guard ensures that when it does, the system fails gracefully rather than burning through all eight steps asking the same question.
- **Final step nudge:** On the last step, the system injects: "You MUST provide your best answer now. Do NOT call another tool." This guarantees the loop terminates with output.
- **Confidence gating:** If confidence is below 0.75 and a remote Skye model is configured, the answer gets escalated for a second opinion.
- **URL validation:** Any URLs in the final answer that weren't observed during tool execution get stripped. The model cannot fabricate links.

Layer 6: System Prompt Assembly

The system prompt is not a static string. It is *assembled* at the start of each agent run from multiple sources:

1. **Identity and rules** — who the model is, the ReAct format, behavioral rules (never invent URLs, call web_search at most once, ignore irrelevant results)
2. **Current date** — injected as `Today: {date}. Year: {year}.` so the model knows what year it is. This sounds obvious. It was not obvious to the model without it.
3. **Current focus** — the reflection engine's latest output, telling the model what topics it's been working on recently
4. **Symbolic hints** — regex-matched guidance for the current query
5. **EEG domain knowledge** — frequency band tables, pipeline steps, software compatibility, tool hints
6. **Tool descriptions** — what each tool does and what parameters it accepts
7. `/no_think` — a directive to suppress Qwen3's thinking tags in output

Layer 7: Answer Formatting & Validation

The raw output from an 8B model is, to put it diplomatically, not always what you'd show a client. The answer formatter handles:

- **JSON leak cleanup:** Sometimes the model wraps its answer in `{"answer": "..."}` instead of providing plain text. The formatter detects ReAct-shaped payloads and extracts the answer field.
- **Dict-to-Markdown conversion:** If the answer is accidentally structured as a Python dict, it gets rendered as readable Markdown sections.
- **URL validation:** Every `[text](url)` link is checked against the set of URLs actually observed during tool execution. If the domain doesn't match any known URL, the link is stripped and only the anchor text remains. This is the anti-hallucination firewall for links.

Tool Catalog

Tools are the agent's hands. Each is a Python class inheriting from `Tool`, auto-discovered by the registry at startup. The registry introspects function signatures so the LLM only sees the parameters each tool actually accepts.

web_search

The web search tool is the most complex tool in the system. A single call triggers a multi-phase pipeline that generates query variants, searches multiple times with staggered execution, scores results by relevance, probes for current-year URLs, fetches full page content, and follows outbound links one hop deep.

▼ Relevance Scoring Algorithm

$$\text{relevance}(\text{item}) = |\{\ \text{kw} \in \text{keywords} : \text{kw} \in (\text{title} \cup \text{body} \cup \text{url})\ \}| \ / \ |\text{keywords}|$$

This is keyword overlap, not semantic similarity. It's fast, it's deterministic, and for web search result ranking it works better than trying to embed search snippets on the fly. A result containing 4 out of 4 query keywords is almost certainly more relevant than one containing 1 out of 4.

▼ Query Variant Generation

Given a user query like "when is the next Suisun Neuroscience Summit?", the system generates:

- Original query (as-is)
- Short keyword form + current year: `suisun summit 2026`
- Event-word extraction: detects "summit," "conference," etc. and builds minimal forms
- **Adjacent keyword concatenation:** If two short keywords appear next to each other (e.g. "sui" + "sun"), they're joined to try "suisun" — catching split-word typos that search engines can't resolve
- Bare keywords without year for broader coverage

▼ URL Probing

When search results contain year-specific paths (e.g. `/suisun-summit-2025/`), the tool generates the current-year equivalent (`/suisun-summit-2026/`) and tests it. It also builds

subdomain probes from query keywords — finding `eegstrategies.com` in results triggers a probe of `suisun.eegstrategies.com`.

▼ **Answer Data Assembly**

All fetched pages are scored with a year-boosted relevance formula:

$$\text{score} = \text{base_relevance} + 0.3 \times [\text{current_year} \in \text{text}] - 0.05 \times [\text{only_prev_year} \in \text{text}]$$

The top sources are ranked and formatted into an `ANSWER DATA` block that tells the LLM: "Here are the facts. Here are the links. Use the best source. Do not search again."

deep_research

When `web_search` is a flashlight, `deep_research` is a searchlight battery. It queries three sources in parallel:

- **DuckDuckGo** — general web results
- **PubMed** — biomedical literature (HTML scraping of search results)
- **Google Scholar** — academic papers (HTML scraping)

Results are fetched, deduplicated by URL, and content is extracted via Trafilatura. PDFs are handled through pypdf. All fetched content is cached in a **SQLite database** with gzip compression and a 48-hour TTL, keyed by MD5 hash of the URL. This means repeated research on the same topic doesn't re-fetch pages.

fetch_url

Single-URL fetch with retry logic and backoff on 429/5xx responses. Shares the deep_research content cache. Handles both HTML (via Trafilatura) and PDF (via pypdf) transparently.

run_python_analysis

This is where the system becomes genuinely dangerous — in the good way. The agent writes a Python script, and the system executes it in a subprocess with:

- Working directory set to `eeg_workspace`

- Matplotlib forced to `Agg` backend (no display server needed)
- Timeout of 300 seconds (configurable)
- **Up to 2 automatic retries** with error-specific fixes: missing ICLabel imports, montage parameter corrections, common MNE API changes
- Output file detection — after execution, it scans the output directory for new files and reports them

The agent can write arbitrary MNE-Python scripts: load EEG files, apply filters, run ICA, compute PSDs, generate topomaps, analyze connectivity — whatever the analysis requires. If the script crashes, the agent reads the traceback, fixes the code, and tries again. Automatically. It is easier to ask forgiveness than to get permission — and easier still to catch the `ModuleNotFoundError`, add the import, and re-run before anyone notices.

run_eeg_pipeline

For when you want the full treatment. Generates and executes a 24-step preprocessing and analysis pipeline. See the EEG Analysis Deep Dive below.

Memory Tools

Tool	Purpose
`query_memory`	Semantic search across all collections; bumps access count and importance on hit
`boost_memory`	Manually mark specific memories as important (increases both importance and manual_boost)
`ingest_document`	Chunk and embed files (PDF, TXT, MD) or EDF/BDF headers into the library
`run_reflection`	Trigger a reflection pass over recent memory

System & Utility Tools

Tool	What It Does
`get_system_stats`	CPU, RAM, disk via psutil; GPU via nvidia-smi CSV parsing
`get_realtime_stats`	Top processes by memory usage, boot time
`get_datetime`	Current date/time with timezone support
`run_shell_command`	Allowlisted presets only: nvidia-smi, netstat, MNE info, pip list, list EEG files

`skye_infer`	Direct query to a remote Ollama instance (optional Skye model)
`send_discord_message`	Post to a Discord channel via REST API

EEG Analysis Deep Dive

EEG — electroencephalography — is the art of listening to the brain's electrical conversation through the skull. It is noisy, it is subtle, and it is one of the few non-invasive windows we have into what billions of neurons are doing at any given moment. The signal you record at the scalp is the sum of postsynaptic potentials from millions of cortical pyramidal cells, attenuated by bone, fluid, and skin, contaminated by eye blinks, muscle tension, line noise, and the occasional loose electrode. Half a million of these recordings, reviewed by clinicians who learned to read the patterns the way a sailor reads the sea, produced the phenotype classifications that inform how this system interprets what it measures. The EEG does not tell you what the DSM says the patient has; it tells you what the cortex is doing. These are not always the same thing, and when they disagree, the cortex is not the one that is wrong.

Paradox's EEG stack is built on **MNE-Python**, the open-source gold standard for M/EEG analysis, extended with autoreject, MNE-ICALabel, and MNE-Connectivity.

The 24-Step Pipeline

Every step is wrapped in try/except. The pipeline continues even if individual stages fail, producing a partial report rather than nothing. In clinical and research contexts, *some* analysis is always better than a crash dump. Silent `except: pass` blocks — the software equivalent of ignoring a flat EEG and calling it "normal" — have been systematically replaced with `logger.warning` so failures are visible. Light may always find the darkness waiting, but it should at least report what it found there.

▼ **Steps 1-3:** Load, Diagnose, Visualize

Step 1: Data Loading

Auto-detects format from extension and dispatches to the correct MNE reader:

Extension	Reader	Source Systems
.edf	`read_raw_edf`	NeuroGuide, BrainMaster, BioExplorer, Cygnet, most clinical EEG

.bdf	`read_raw_bdf`	BioSemi, OpenBCI (24-bit)
.fif	`read_raw_fif`	MNE native, Elekta/MEGIN
.set	`read_raw_eeglab`	EEGLAB (MATLAB)
.vhdr	`read_raw_brainvision`	Brain Products (actiCHamp)
.cnt	`read_raw_cnt`	Neuroscan
.gdf	`read_raw_gdf`	General Data Format

Attempts to apply the International 10-20 montage automatically. If channel names don't match standard positions, it warns but continues.

Step 2: Quality Diagnostics

Computes per-channel **variance**, **kurtosis** (via SciPy), and **peak-to-peak range**. Channels with variance below 10^{-12} are flagged as flatlines — dead electrodes producing no signal.

Step 3: Baseline PSD Visualization

Computes and plots the power spectral density of the raw (unprocessed) data. This is your "before" picture — you'll compare it to the cleaned PSD at Step 22.

▼ **Steps 4-8:** Filtering & Channel Cleaning

Step 4: Bandpass Filter

FIR filter with default passband **0.5 – 40 Hz**. The high-pass at 0.5 Hz removes slow drifts (sweat artifacts, electrode motion). The low-pass at 40 Hz attenuates EMG contamination. MNE uses a windowed-sinc FIR by default — phase-linear, no group delay distortion.

Step 5: Notch Filter

Removes power line interference at **60 Hz** (North America) or 50 Hz (configurable). If the first harmonic falls within the low-pass cutoff, it's notched too.

Step 6: Bad Channel Detection

$$\text{bad if: variance(ch)} > 10 \times \text{median(variance)} \textbf{ or } \text{variance(ch)} < 0.01 \times \text{median(variance)}$$

Channels with abnormally high variance are noisy (loose electrode, bridging). Channels with abnormally low variance are dead or saturated. The 10× and 0.01× thresholds are deliberately conservative — you'd rather interpolate one extra channel than let a noisy one corrupt your ICA.

Step 7: Spherical Spline Interpolation

Bad channels are reconstructed from their neighbors using MNE's spherical spline interpolation (Perrin et al., 1989). Requires electrode positions (montage).

Step 8: Average Re-reference

All channels are re-referenced to the common average. This approximates a reference-free recording and is the standard for QEEG and connectivity analysis. The math is simple:

$$V'_i(t) = V_i(t) - (1/N)\,\Sigma_j\, V_j(t)$$

▼ **Steps 9-12:** ICA Artifact Removal

Step 9: Matrix Rank

Computes the numerical rank of the data matrix via SVD. This determines the maximum number of ICA components — you cannot extract more independent sources than the rank of your data.

Step 10: ICA Decomposition

Fits **FastICA** (default) to decompose the signal into maximally statistically independent components. Each component is a spatial pattern (weights across channels) times a time course. Eye blinks, heartbeat, and muscle artifacts typically separate into distinct components because they're generated by independent physical sources.

$$X = A \cdot S \rightarrow S = W \cdot X$$

Where X is the data matrix, A is the mixing matrix, S is the source matrix, and $W = A^{-1}$ is the unmixing matrix found by maximizing non-Gaussianity of the sources.

Step 11: Component Classification (ICLabel)

Uses **MNE-ICALabel** — a convolutional neural network trained on thousands of labeled ICA components — to classify each component as: Brain, Eye, Heart, Muscle, Line Noise,

Channel Noise, or Other.

Components classified as non-brain with probability ≥ **0.80** (configurable) are marked for exclusion. If ICLabel is unavailable, falls back to EOG correlation (find components correlated with eye channels).

Step 12: Signal Reconstruction

Excluded components are removed and the signal is reconstructed:

$$X_clean = A_brain \cdot S_brain$$

Where only the brain-classified columns of A and rows of S are used in the multiplication.

▼ **Steps 13-17:** Epochs & ERPs

Step 13: Event Detection

Finds stimulus markers in the recording. Falls back to annotations if no event channel exists.

Step 14-15: Epoching & Rejection

Data is segmented into time-locked windows around events (default −200ms to +800ms). Epochs exceeding **150 µV** peak-to-peak are rejected. If fewer than 3 events exist, falls back to fixed-length 2-second epochs.

Step 16: ERP Averaging

The Event-Related Potential is the average waveform across all epochs at each channel. By averaging, random noise cancels out and the time-locked neural response emerges:

$$ERP(t) = (1/N)\ \Sigma_i\ epoch_i(t)$$

The signal-to-noise ratio improves as √N — 100 epochs gives 10× better SNR than a single trial.

Step 17: ERP Image

Generates a 2D image with trials on the y-axis and time on the x-axis, color-coded by amplitude. Useful for spotting trial-by-trial variability and systematic drifts.

▼ **Steps 18-21:** Spectral & Connectivity Analysis

Step 18: Time-Frequency (Morlet Wavelets)

Decomposes the signal into time-varying frequency content using **Morlet wavelets** — complex sinusoids windowed by a Gaussian. The number of cycles scales with frequency (n_cycles = freq/2), giving better frequency resolution at low frequencies and better time resolution at high frequencies.

$$\psi(t, f) = \exp(2\pi i f t) \cdot \exp(-t^2 / 2\sigma^2) \text{ where } \sigma = n_cycles / (2\pi f)$$

Step 19: Spectral Band Power (Welch's Method)

Welch's method computes the PSD by dividing the signal into overlapping segments, windowing each (Hamming), computing the FFT, squaring the magnitudes, and averaging:

$$PSD(f) = (1/K)\ \Sigma_k\ |FFT(x_k \cdot w)|^2 / (fs \cdot \Sigma\ w^2)$$

Band power is computed by integrating the PSD over each frequency range:

$$Power_band = \Sigma\ PSD(f) \text{ for } f \in [f_low, f_high]$$

Band	Range	Clinical Association
Delta	0.5 – 4 Hz	Deep sleep, cortical inhibition, healing
Theta	4 – 8 Hz	Drowsiness, meditation, memory encoding
Alpha	8 – 13 Hz	Relaxed wakefulness, posterior dominant rhythm
Beta	13 – 30 Hz	Active thinking, focus, anxiety (excess)
High Beta	20 – 30 Hz	Anxiety, hypervigilance, rumination (subset of beta)
Gamma	30 – 40 Hz	Binding, higher cognition, perception

v0.3.1 note: Band ranges are now *unified* across all modules — `band_definitions.py`, `spectrum_generator.py`, the orchestrator, and NetOps copies all agree on `beta=(13,30)`, `hibeta=(20,30)`, `gamma=(30,40)`. Prior to this, inconsistencies meant that "beta power" could mean three different things depending on which code path computed it. When you compare a patient's recording to a normative database, the patient and the

database need to be speaking the same frequency language — otherwise you are comparing the electrical equivalent of apples to slightly different apples, and calling the difference "abnormal."

Topomaps are generated for each band — spatial maps showing the distribution of power across the scalp.

Step 20: Functional Connectivity

Computes **spectral coherence** (default) between all channel pairs using `mne-connectivity`:

$$\mathrm{Coh}(f) = |S_xy(f)|^2 \,/\, (S_xx(f) \cdot S_yy(f))$$

Where S_xy is the cross-spectral density and S_xx, S_yy are auto-spectral densities. Coherence ranges from 0 (independent) to 1 (perfectly phase-locked). Alternative methods available: PLI (Phase Lag Index), PLV (Phase Locking Value).

Step 21: Network Metrics

$$\mathrm{Node_strength}(i) = \Sigma_j\, \mathrm{Con}(i,j) \,/\, (N - 1)$$

$$\mathrm{Global_synchronization} = \mathrm{mean}(\mathrm{upper_triangle}(\mathrm{Con}))$$

Reports the top 5 most strongly connected channel pairs. High global synchronization can indicate over-coupling (epileptiform activity, medication effects). Low values may indicate disconnection syndromes.

▼ **Steps 22-24:** Quality Summary & Export

Step 22: Clean PSD

Computes PSD of the cleaned signal for comparison with the raw baseline from Step 3.

Step 23: Export

Saves cleaned raw data (.fif), ICA weights (.fif), and epochs (.fif) to the output directory.

Step 24: Report & Metrics

Generates a human-readable text report and a machine-readable JSON metrics file containing all quality metrics, step statuses, and processing parameters.

Signal Processing Mathematics

Fourier Transform & PSD

The Discrete Fourier Transform converts time-domain signals to frequency-domain:

$$X(k) = \Sigma_n x(n) \cdot e^{-j2\pi kn/N}$$

The Power Spectral Density is the squared magnitude of the Fourier coefficients, normalized by the sampling rate and window function. Welch's method reduces variance by averaging multiple windowed segments.

Independent Component Analysis

ICA assumes the observed signal X is a linear mixture of statistically independent sources S:

$$X = A \cdot S$$

FastICA finds the unmixing matrix W by maximizing the non-Gaussianity of the estimated sources, typically using negentropy as the contrast function:

$$J(y) = [E\{G(y)\} - E\{G(\nu)\}]^2$$

Where G is a non-quadratic function (e.g., log cosh), y is the estimated source, and ν is a standard Gaussian. The Central Limit Theorem guarantees that the mixture is more Gaussian than any individual source, so maximizing non-Gaussianity recovers the original independent components.

Morlet Wavelet Transform

The Morlet wavelet is a complex exponential modulated by a Gaussian envelope:

$$\psi(t) = \pi^{-1/4} \cdot e^{j2\pi f_0 t} \cdot e^{-t^2/2}$$

Convolution with the signal gives instantaneous power and phase at each time-frequency point. The trade-off between time and frequency resolution is controlled by the number of cycles parameter — more cycles means better frequency resolution but worse time resolution.

EEG File Formats

Format	Extension	Resolution	Key Features
European Data Format	.edf	16-bit	Universal interchange; most systems export it
BioSemi Data Format	.bdf	24-bit	Higher dynamic range; EDF+ variant
MNE/Elekta FIF	.fif	Float	Native MNE; stores raw, epochs, evoked, forward, inverse
BrainVision	.vhdr	Various	Triplet: .vhdr (header) + .vmrk (markers) + .eeg (data)
EEGLAB	.set	Various	MATLAB structure; data in .fdt companion file

Software Ecosystem Compatibility

Software	Export Format	Typical Use	MNE Loader
NeuroGuide	EDF	QEEG, z-scores, sLORETA	`read_raw_edf`
BrainMaster	EDF/BDF	Discovery, Atlantis, BrainAvatar	`read_raw_edf/bdf`
BioExplorer	EDF, CSV	Visual neurofeedback design	`read_raw_edf`
BioEra	EDF	Custom biofeedback protocols	`read_raw_edf`
Cygnet	EDF	Z-score neurofeedback with norms	`read_raw_edf`
OpenBCI	CSV, BDF	Cyton, Ganglion via BrainFlow	`read_raw_bdf` / pandas

EEGLAB	.set/.fdt	MATLAB research ecosystem	`read_raw_eeglab`
Brain Products	.vhdr	actiCHamp, BrainVision	`read_raw_brainvision`
Elekta/MEGIN	.fif	MEG + EEG combined	`read_raw_fif`

Interactive Outputs, API Jobs & NetOps Pipeline

Beyond the 24-step script, Paradox runs a **visualization orchestrator** (`hexnode/eeg/viz/run_visualizations.py`) after automated preprocessing. It loads cleaned FIF when present (else the original recording), computes per-channel spectra for topomap generators, builds **microstate** figures and interactive HTML, and invokes the **NetOps standalone** bundle (`hexnode/eeg/netops_runner.py` → `netops_standalone/pipeline.py`): Granger-style connectivity, multi-policy traceroute, extended analyses, and dozens of standalone HTML dashboards merged into the primary **traceroute interactive** explorer via a dropdown.

The **FastAPI** app exposes EEG operations the web UI consumes:

- `POST /eeg/process` — multipart upload; spawns a background thread that runs the generated pipeline, clinical helper scripts (e.g. Clinical Q, band power), then the visualization subprocess. Each job writes into `data/eeg_workspace/output/<job_id>/` with persisted `_job.json` state.
- `GET /eeg/jobs`, `GET /eeg/jobs/{id}` — list and poll status, messages, and output filenames.
- `GET /eeg/jobs/{id}/files/{path}` — serve artifacts; HTML responses use iframe-friendly headers for in-app preview.
- `GET /eeg/outputs` / `GET /eeg/outputs/{file}` — browse the flat global output directory (legacy or non-job files).
- `POST /workspace/open` — open the output folder in the OS shell (Explorer on Windows).

The **Next.js EEG Data panel** drives upload, polling, and embedded HTML preview. Full operator narrative: `doc/user/09-eeg-research.md`.

The Bundled EEG Worker (v0.3.1)

The most dangerous phrase in the old architecture was "install a proper venv with `[eeg]` extras." It was fine for developers, but for a clinician who wants to process an EDF and go home, it was the equivalent of asking them to build their own oscilloscope before measuring a

signal. Humans are allergic to change — but they are *really* allergic to debugging pip dependency trees.

As of v0.3.1, the desktop installer bundles `paradox-eeg-worker.exe`: a dedicated PyInstaller-frozen executable that carries the full science stack — MNE-Python, SciPy, Matplotlib, Plotly, scikit-learn, networkx, statsmodels, and every transitive dependency they drag in. It lives at `dist/paradox-api/eeg-worker/paradox-eeg-worker.exe`, beside the lean API bundle.

How the Worker Runs

The entry point is `eeg_subprocess_launcher.py`, which handles three tasks:

1. **PYTHONPATH injection** — PyInstaller executables do not respect the `PYTHONPATH` environment variable at startup. The launcher calls `_inject_pythonpath()` to splice the API bundle's `_internal` directory onto `sys.path`, so `import hexnode.eeg.*` resolves correctly even though the worker was compiled separately.
2. `--eeg-probe` **self-test** — Called by `GET /health` to verify the worker can start, import MNE, and exit cleanly. Timeout: 60 seconds (increased from 25s — the first boot on a cold disk is not the time for impatience).
3. `runpy` **dispatch** — Runs the target script (`run_visualizations.py`, pipeline scripts, clinical scripts) in the worker process with the correct working directory and Matplotlib `Agg` backend.

Resolution Order

The function `python_for_eeg()` in `hexnode/config.py` resolves the interpreter in this order:

1. **Bundled worker** — `eeg-worker/paradox-eeg-worker.exe` relative to the frozen API executable (preferred when `sys.frozen` is set)
2. `PARADOX_EEG_PYTHON` / `EEG_PYTHON` — explicit override via environment variable
3. **Discovery** — `py -3` on Windows, then `python3`, then `python`

The health endpoint reports exactly which path was chosen, whether the bundled worker was found, and whether `run_visualizations.py` exists inside the bundle. Check `GET /health` → `eeg_subprocess` — one accurate measurement beats speculation.

Clinical Summaries, Vigilance & Pattern–Condition

The NetOps pipeline attaches **per-site metrics** (band power, relative power, ratios), **wPLI** and related connectivity, **slow-wave** proxies, and a **vigilance** block: primary path uses a windowed VIGALL-inspired classifier; a slow-wave-derived fallback fills global state when needed so reports and guards stay populated.

Pattern–condition analysis (`cracker.pattern_condition`) ingests markers from metrics, connectivity, and vigilance, evaluates YAML registry rules, and feeds narrative sections in HTML and the agent context.

Clinician summary (`clinician_summary_session.html`) is a multi-tab standalone page: Easy / Advanced / Expert views, named methodology tabs (Gunkelman, Swingle ClinicalQ, Fisher, Thatcher, Lubar, Peniston, Sterman, Hammond, Budzynski, Fehmi), driven by flattened per-site metrics and session-relative z where normative keys are absent. It is *educational and screening-oriented*, not a standalone medical device output.

Additional linked dashboards (non-exhaustive): latency SLA tiers, packet-loss heatmaps, microstate–propagation coupling, PEM, topology/path analytics, runbooks, evidence summaries, TBI/PD-oriented summaries, dedicated vigilance timeline HTML, and pattern–condition evidence panels — exact set depends on successful module import inside the standalone bundle.

Phenotype-Informed Analysis

Over half a million EEGs, reviewed by hand, reveal a finite set of recurring electrical signatures — patterns that cut across DSM categories and point instead at what the cortex is actually *doing*. The phenotype model, refined over decades of clinical QEEG practice, identifies eleven candidate clusters that represent intermediate phenotypes between genetics and behavior. Paradox references this framework throughout its clinical outputs, not because software can replace the trained eye that classified those half-million recordings, but because the patterns themselves are measurable — and what can be measured can be compared to a normative database, which is considerably more reliable than memory.

▼ **The 11 Candidate EEG Phenotypes**

#	Phenotype	Clinical Signature
1	Persistent Alpha (Eyes Open)	Alpha rhythm fails to attenuate with eyes open; posterior dominant rhythm persists under task load

2	Low Voltage (Fast or Slow)	Generally low magnitudes across the scalp; may be fast-dominant or slow-dominant variant
3	Spindling Excessive Beta	Prominent rhythmic beta activity, often medication-related (benzodiazepines, barbiturates) or constitutional
4	Faster Alpha Variants	Alpha peak frequency shifted above typical 10 Hz; not low voltage
5	Epileptiform	Spikes, sharp waves, spike-and-wave complexes; requires clinical correlation
6	Excess Temporal Alpha	Alpha activity prominent at temporal sites beyond normal posterior distribution
7	Frontal Asymmetries	Lateralized power differences at frontal sites; associated with mood and motivation
8	Frontal Lobe Disturbances	Excess slow activity or disorganization at frontal sites
9	Mixed Fast and Slow	Concurrent excess in delta/theta and beta; "busy but inefficient" cortex
10	Focal Abnormalities (Non-Epileptiform)	Localized slowing or amplitude asymmetry without spike morphology
11	Diffuse Slow Activity	Generalized excess delta/theta with or without slower alpha peak; metabolic, medication, or degenerative processes

These phenotypes inform the **clinician summary** tabs and the **pattern–condition** analysis engine. The system does not diagnose — it measures, compares to norms, and presents the patterns. The clinician connects the measurement to the patient. Targeting treatment by phenotype rather than symptom checklist has been shown to raise SSRI response rates from roughly 35% to approximately 80%: the EEG tells you what the brain is doing, which turns out to be rather more useful than asking the brain's owner what they think it is doing.

Advanced Visualizations (v0.3.1)

The visualization orchestrator gained four new output families in this release. Each is generated when the requisite data is available; missing prerequisites result in a logged warning rather than a crash, because in clinical and research contexts some analysis is always better than none

— build a clinician a partial report and they'll glean insight for a session; crash the pipeline and they'll glean nothing at all.

▼ Coben-Style Granger Brain Overlays

When Granger causality results are available from the NetOps pipeline, the orchestrator generates a family of brain-overlay visualizations in the style used in directed connectivity research:

- **3D brain overlay** — directed arrows between electrode sites, color-coded by Granger F-statistic magnitude, rendered on a translucent cortical surface
- **Slice views** — axial, sagittal, and coronal projections of the connectivity graph
- **Time-resolved Granger** — windowed causality evolution showing how directional influence between regions changes across the recording
- **Spectral Granger matrix** — band-by-band causality heatmap (delta through gamma) for all channel pairs

These are generated as interactive Plotly HTML and static PNG. The directed nature of Granger — A predicts B is not the same as B predicts A — makes it particularly useful for identifying driver sites and propagation pathways in dysregulated networks.

▼ LORETA Source Localization

When raw MNE `Raw` data is available (stored in results as `_raw_mne` since v0.3.1), the pipeline generates source-localization estimates using sLORETA/eLORETA-style inverse solutions. The visualization shows estimated cortical current density projected onto standardized brain anatomy — answering not just "what frequency is abnormal?" but "where in the cortex is the generator?"

Source localization from scalp EEG is an *ill-posed inverse problem* (infinite source configurations can produce the same scalp distribution), but regularized minimum-norm estimates — particularly the standardized variant — provide clinically useful spatial priors. The output is explicitly labeled as an estimate, not a ground truth: the map shows where the math *thinks* the generators are, which is useful, but not the same as looking directly at the cortex.

▼ Mahalanobis Distance 3D Phenotype

When z-score metrics are available from normative comparison, the system computes the **Mahalanobis distance** — a multivariate measure of how far an individual's feature vector is from the normative centroid, accounting for correlations between features:

$$D_M = \sqrt{(\ (x - \mu)^T\ \Sigma^{-1}\ (x - \mu)\ )}$$

Unlike simple z-scores (which treat each band and channel independently), Mahalanobis distance captures the *joint* deviation: a patient with moderately elevated theta *and* moderately reduced alpha may have a perfectly normal z-score on each metric individually, but a very abnormal Mahalanobis distance because the combination is rare. The visualization renders this as a 3D phenotype surface — a topographic landscape where peaks represent regions of maximal statistical distance from the normative population.

▼ Trace Viewer

An interactive, browser-based EDF trace viewer: scrollable multi-channel time-series display with amplitude scaling, channel reordering, and zoom — the digital equivalent of the endless paper roll that EEG technologists have been reading since Berger's first recordings in 1924. Generated as a self-contained HTML file using Plotly, viewable in any browser, embeddable in the job output iframe.

The trace viewer is generated when raw data is available. It complements the processed topomaps and spectra by letting the clinician see the *actual waveforms* — because sometimes the most important feature in a recording is a single transient that averages out in the spectral analysis but is obvious to a trained eye scrolling through the trace at 30mm/sec.

The Reflection Engine

Most AI assistants have the long-term memory of a startled mayfly. Paradox has a reflection engine — a periodic process that reviews recent memories, identifies patterns, and writes a "current focus" document that steers future interactions. Like a clock running counter-clockwise

on the wall, it exists to remind the system that what happened yesterday matters to what happens tomorrow — and that the default state of software is to forget everything the moment you restart it.

How It Works

1. Two rounds of semantic queries sample diverse content from each memory collection (chat_history, documents, library).
2. The samples are bundled (capped at ~16K chars) and sent to the LLM with a structured prompt asking for: key themes, knowledge gaps, recurring topics, and a concise focus statement.
3. The LLM returns JSON with these fields plus a confidence score.
4. If confidence is above `0.35`, the focus is written to `current_focus.md`. If below, the previous focus is preserved — the system doesn't downgrade its own orientation based on a bad reflection pass.
5. The full reflection is saved as a timestamped markdown file and embedded into the documents collection.

The focus document is injected into every subsequent system prompt, giving the model a sense of "what I've been doing lately." Over time, this creates a feedback loop: the model remembers patterns, the reflection engine crystallizes them, and the crystallized focus guides future reasoning.

> *It is not consciousness. It is not even close. But it is a memory system that knows what it has been thinking about, and that is more than most software can claim. A man is not dead while his name is still spoken; a memory is not lost while its importance score is still above the decay threshold.*

Version Tracking

A program that does not know what version it is cannot be trusted to know what version of anything else it is running, either. As of v0.3.1, the version is a single source of truth:

File / Script	Purpose
`VERSION`	Plain-text file at repo root containing `0.3.1` (or current version). Read by build scripts, Tauri config, and `pyproject.toml`.

`CHANGELOG.md`	Keep-a-Changelog format. Every fix, feature, and documentation change is recorded here — because memory is what separates a project from a collection of files that happen to be in the same directory.
`scripts/bump_version.ps1`	Updates `VERSION`, `pyproject.toml`, `tauri.conf.json`, and `Cargo.toml` in one pass. Takes a version string argument. Refuses to run without one, because assuming a default version is how you ship 0.0.0 to production.

The `VERSION` file is read by `build_release.ps1` to stamp installer filenames. The Tauri config reads it for the window title and about dialog. The Python package reads it in `pyproject.toml`. One number, one file, one truth. The rest is derivation.

> *A name is not dead while it is still spoken. A version is not dead while it is still tracked.*

Frontend & HUD

The frontend is a Next.js application with a dark aesthetic that can only be described as "control room chic." Two HUD themes: **cyan** (the default, for when you want to feel like you're piloting a submarine) and **matrix** (green on black, for when you want to feel like you're in a movie about piloting a submarine).

Components

- **Chat panel:** Messages render as full Markdown via `react-markdown` + `remark-gfm`. Links are clickable (target _blank). Tables, code blocks, lists, and blockquotes all render properly.
- **System sidebar:** Ollama health status (polling every 15 seconds), GPU optimization badges (`FA+q8_0`), GPU/RAM/CPU stats as JSON, current focus text, and a semantic memory search panel. Includes an "GPU Optimizations" section showing Flash Attention, KV cache type/savings, and embedding quantization mode.
- **License panel:** Toggle from the header badge. Shows active license info (organization, tier, features, expiration) or an activation form (machine ID display, key paste, activate button).
- **File panel:** `FilePanel` component with drag-and-drop upload zone, category-filtered file list, and delete buttons. Supports EEG, documents, and general files.

- **EEG Data panel:** Upload recordings, start `/eeg/process` jobs, poll progress, list artifacts, preview interactive HTML (traceroute, clinician summary, etc.) in an iframe, and open the job output folder on disk.
- **Header badges:** Health status, optimization indicator (`FA+q8_0`), license tier, HUD theme toggle.
- **Theme toggle:** Persisted in localStorage as `paradox-hud-color`. Sets `data-hud` attribute on the document root; CSS custom properties handle the rest.

Tauri Detection

The frontend detects `window.__TAURI_INTERNALS__` to determine if it's running inside the desktop app. When in Tauri mode, the API base URL is hardcoded to `http://127.0.0.1:8765` and health check failures (2+ consecutive) trigger a redirect to the `/setup` page.

API Client

The frontend talks to the backend through functions in `api.ts`: agent, health, stats, focus, memory query, license management, and file operations — all pointing at `http://127.0.0.1:8765` by default.

REST API

Method	Endpoint	Purpose
GET	`/health`	Status, Ollama connectivity, optimization info, license status, `eeg_subprocess` (bundled worker path, `bundled_worker`, `viz_script_found`), `eeg_norms_addon`
POST	`/agent`	Main agent endpoint — send message, receive answer. Returns **504** on Ollama read timeout, **502** when Ollama is unreachable, **503** when no models loaded.
POST	`/memory/query`	Semantic search across memory collections
GET	`/system/stats`	CPU, RAM, disk, GPU metrics
GET	`/focus`	Current reflection focus document
GET	`/tools`	List registered tools (filtered by license)

POST	`/ingest/path`	Ingest a file by filesystem path
POST	`/files/upload`	Upload files (EEG, documents, general)
GET	`/files`	List uploaded files (with category filter)
DELETE	`/files/{cat}/{name}`	Delete an uploaded file
GET	`/license/status`	License validity, features, expiration
GET	`/license/machine`	Machine ID and system info
POST	`/license/activate`	Activate a license key
POST	`/license/deactivate`	Remove active license
GET	`/eeg/outputs`	List global EEG output files (HTML, images, JSON)
GET	`/eeg/outputs/{filename}`	Download one file from global output
POST	`/eeg/process`	Upload EEG; start background pipeline + viz job
GET	`/eeg/jobs`	List processing jobs
GET	`/eeg/jobs/{id}`	Job status, messages, output file list
GET	`/eeg/jobs/{id}/files/{path}`	Serve job artifact (iframe-safe HTML)
POST	`/eeg/jobs/{id}/delete`	Delete job directory
POST	`/workspace/open`	Open EEG output folder in OS file manager

All endpoints return JSON. The `/agent` response includes: `answer`, `confidence` (0-1), `steps` (array of thought/action/observation records), `trace_id`, `escalated_skye` (boolean), and `symbolic_hints` (if triggered). The `/health` response includes an `optimizations` object with flash attention, KV cache type/savings, and embedding quantization status, plus an `eeg_subprocess` block reporting: resolved `python` path, `frozen` flag, `bundled_worker` (true when `paradox-eeg-worker.exe` is found), `viz_script_found`, and any `warning` / `error` from the probe.

The `GET /eeg/jobs` listing now includes `condition` and `output_mode` fields per job, so the frontend can display badge labels without polling each job individually. This is the kind of optimisation that saves milliseconds per request — which, laid end to end like nanosecond wires, add up to a user experience that does not make people tap their desks.

GPU & Memory Optimization

Running a competent language model on consumer hardware is a game of inches. Every byte of VRAM matters. Every millisecond of memory bandwidth counts. Paradox applies three layers of optimization — one for how the model *thinks*, one for how it *remembers what it's thinking*, and one for how it *stores what it's thought about*.

Target Hardware

Component	Spec	Role
RAM	32 GB	ChromaDB, Python analysis, general headroom
GPU	RTX 5060 (8 GB VRAM)	Ollama inference (Qwen3:8B + phi4-mini)
Storage	SSD	ChromaDB, EEG workspace, model weights

Flash Attention

Standard attention computes the full N×N attention matrix, which for a 16K context means 256 million entries — each a float16. Flash attention rewrites this as a series of tiled operations that never materialize the full matrix, keeping everything in fast SRAM instead of slow HBM (GPU global memory).

Configuration

```
# Environment variable (set by Tauri on Ollama launch, or in .env)
OLLAMA_FLASH_ATTENTION=1

# Also passed per-request via Ollama API options:
{"options": {"flash_attn": true}}
```

Effect: Reduces attention VRAM from $O(N^2)$ to $O(N)$, allowing longer context windows on the same hardware. Also delivers ~2x speedup on attention computation because the memory-bound bottleneck is eliminated.

KV Cache Quantization

During inference, the model stores the key and value tensors for every token in the context. For Qwen3:8B with 16K context, this cache alone consumes approximately **2 GB** of VRAM at float16. KV cache quantization compresses these tensors:

Mode	Bits per Value	VRAM Savings	Quality Impact
`f16` (default)	16	None (baseline)	None
`q8_0` recommended	8	~50%	Negligible
`q4_0`	4	~75%	Small to medium

Configuration

```
# Environment variable
OLLAMA_KV_CACHE_TYPE=q8_0

# In .env (Paradox config)
OLLAMA_KV_CACHE_TYPE=q8_0
```

With `q8_0`, the KV cache for 16K context drops from ~2 GB to ~1 GB. Combined with flash attention, this frees enough VRAM to push the context to 32K or even 64K on an 8 GB GPU — meaning longer conversations and deeper agent reasoning chains without out-of-memory crashes.

Embedding Quantization

Every memory entry in ChromaDB stores a 768-dimensional float32 embedding vector — that's **3,072 bytes per vector**. Over thousands of documents and conversation turns, this adds up. The embedding quantizer in `hexnode/embed_quantize.py` compresses these vectors before storage.

The Algorithm

1. **Random rotation:** A deterministic orthogonal matrix R (generated via QR decomposition with seed 42) is applied to the raw embedding. This spreads information uniformly across all dimensions — a data-oblivious step that works on any input without calibration.
2. **Scalar quantization:** Each coordinate of the rotated vector is mapped to a discrete set of levels (256 for int8, 16 for int4, 2 for binary).

3. **Inverse rotation:** R^1 is applied to map back to the original coordinate space. ChromaDB receives a vector that looks like a normal embedding but contains quantized information.

$$\hat{v} = R^T \cdot Q(R \cdot v)$$

Where *Q* is the scalar quantizer and *R* is the rotation matrix. The key insight: the rotation makes coordinates approximately independent and identically distributed, so per-coordinate quantization is near-optimal without any data-specific training.

Bit Width	Levels	Effective Compression	Search Quality
32 (off)	Continuous	1x (baseline)	Perfect
8 default	256	~4x	Excellent (>99% recall)
4	16	~8x	Good (~95% recall)
1 (binary)	2	~32x	Acceptable for coarse search

Configuration

```
# In .env
EMBED_QUANTIZE_BITS=8  # 0=off, 8=int8, 4=int4, 1=binary
```

TurboQuant: The Theory Behind It

The embedding quantization module is inspired by **TurboQuant** (Google Research, ICLR 2026), a data-oblivious vector quantization algorithm that achieves near-optimal compression.

▼ Data-Oblivious vs. Data-Dependent Quantization

Traditional methods like **Product Quantization (PQ)** require a training step: they run k-means clustering on your data to build a codebook. This takes hundreds of seconds for large datasets, and the codebook is specific to the data distribution — change the data, retrain the codebook.

TurboQuant eliminates this entirely. A random rotation transforms any input distribution into a predictable form (concentrated Beta distribution on each coordinate). Because the

math works regardless of input, it requires **zero preprocessing**. This is why we use it for a live system that ingests documents continuously — there's nothing to retrain.

▼ Inner Product Bias Correction

In the LLM's attention mechanism, inner products determine which tokens are important. Traditional compression (optimized for Mean Squared Error) systematically shrinks these values — a tiny bias that, multiplied across billions of parameters, causes accuracy degradation.

TurboQuant's full algorithm uses a two-stage approach: compress the main data, then apply a **Quantized Johnson-Lindenstrauss (QJL)** transform to the residual, creating an unbiased estimator. Our embedding quantizer implements the first stage (rotation + scalar quantize); the residual correction becomes relevant when KV cache quantization advances beyond q8_0.

▼ Shannon's Lower Bound

Information theory defines a minimum distortion for any given compression rate — the **rate-distortion function**. TurboQuant is provably within **2.7×** of this absolute mathematical limit across all bit-widths. For 1-bit compression, it is only 1.45× away from perfection.

$$D(R) \geq \sigma^2 \cdot 2^{-2R} \text{ (Shannon lower bound for Gaussian source)}$$

Where D is distortion, R is rate in bits per dimension, and σ^2 is the source variance. TurboQuant achieves $D \leq 2.7 \cdot D_{optimal}$, which for a data-oblivious method is remarkable.

Future: When TurboQuant Lands in Ollama

The llama.cpp community has already built TurboQuant implementations. When this lands in Ollama as a new `kv_cache_type` option, the upgrade path is a single environment variable change — from `q8_0` to a TurboQuant-based 3-bit mode that achieves **6× compression with zero accuracy loss**. The Paradox config layer is already designed for this: change `OLLAMA_KV_CACHE_TYPE` and restart.

Desktop Application

Not everyone wants to wrangle Python virtual environments and terminal commands. The desktop application wraps the entire Paradox stack into a single installable package that handles everything from downloading Ollama to pulling models to starting the backend — all through a guided first-run wizard.

Tauri v2 Shell

The desktop shell is built with **Tauri v2** (Rust). Unlike Electron, Tauri uses the system's native WebView — no bundled Chromium, no 200 MB of browser engine. The Tauri binary itself is ~5 MB; the bulk of the installer size is the PyInstaller backend bundle.

Component	Technology	Size
Desktop shell	Tauri v2 (Rust)	~5 MB
Frontend	Next.js static export	~2 MB
Backend sidecar (API)	PyInstaller onedir bundle	~246 MB
EEG worker	PyInstaller onedir (inside API bundle)	~300 MB
NSIS installer	Compressed package (total)	~340 MB

System Tray Integration

Closing the window minimizes to the system tray — the backend keeps running. Right-click the tray icon for "Show Window" or "Quit Paradox" (which kills the sidecar process and exits).

Sidecar & Worker Architecture

The Python FastAPI backend is bundled by PyInstaller into a standalone executable directory. Tauri manages it as a child process:

1. On startup, Tauri searches multiple candidate paths for `paradox-api.exe` (resource directory, parent directories — accounting for NSIS/MSI installation layouts).
2. The sidecar is spawned with its working directory set to the bundle directory (critical: PyInstaller's `_internal/` directory must be discoverable).
3. Tauri polls `http://127.0.0.1:8765/health` until the backend is healthy (up to 60 seconds).
4. On quit, Tauri sends `taskkill /F /PID` to terminate the sidecar.

Key PyInstaller configuration: the API spec (`paradox-api.spec`) uses a comprehensive `_EXCLUDES` list to keep the API bundle lean — no PyTorch, no Transformers, no scipy, no matplotlib. The heavy science stack ships separately in **`paradox-eeg-worker.exe`** (built from `paradox-eeg-worker.spec`), which lives at `eeg-worker/` inside the API bundle directory. This separation is deliberate: the API starts in under 3 seconds; the worker loads the full MNE/SciPy/Plotly universe only when an EEG job actually needs it. Go ahead and do it — but do it in the right process.

The Two-Executable Design

Executable	Contents	Approx. Size	When It Runs
`paradox-api.exe`	FastAPI, httpx, ChromaDB, agent loop, tool registry	~246 MB	Always (sidecar)
`paradox-eeg-worker.exe`	MNE, SciPy, Matplotlib, Plotly, scikit-learn, networkx, statsmodels	~300 MB	On demand (subprocess per EEG job)

The worker inherits `PYTHONPATH` pointing at the API's `_internal` directory so both executables resolve `import hexnode` to the same source tree. This was the hardest bug in the v0.3.1 cycle — PyInstaller frozen exes ignore `PYTHONPATH` at startup, so the launcher manually injects the path via `sys.path` before any hexnode imports. It is easier to ask forgiveness than permission, but it is easier still to inject the path correctly on the first try.

Build Pipeline (v0.3.1)

Two scripts build the entire release. If you are thinking "surely there are more scripts than that," you are thinking of the old way. We do not do things the old way. The clock on the wall runs counter-clockwise.

`scripts/build_backend.ps1`

1. Builds `paradox-eeg-worker.exe` from `paradox-eeg-worker.spec` (the heavy science bundle)
2. Builds `paradox-api.exe` from `paradox-api.spec` (the lean API bundle)
3. Copies the worker into `dist/paradox-api/eeg-worker/`

4. Validates the full bundle (worker exists, `_internal` present, key files intact)

If the old `dist/paradox-api` directory cannot be removed (PermissionError — something is holding a file handle), the script renames it out of the way and fails loudly if that doesn't work. Silent failures during packaging are the digital equivalent of a loose electrode: you won't know the data is wrong until the report looks absurd.

`scripts/build_release.ps1`

1. Runs `npm run build` in `web/` for the static Next.js export
2. Delegates to `build_backend.ps1` for both executables
3. Runs `cargo tauri build --ci` to produce MSI and NSIS installers

Output lands in `src-tauri/target/release/bundle/msi/` and `bundle/nsis/`, with version numbers sourced from the `VERSION` file.

First-Run Setup Flow

The setup page (`/setup`) orchestrates these Tauri commands:

1. `check_ollama` — Searches standard install paths and PATH for `ollama.exe`.
2. `download_ollama` — If missing, downloads `OllamaSetup.exe` from ollama.com and runs it silently.
3. `ensure_ollama_serving` — Spawns `ollama serve` with `OLLAMA_FLASH_ATTENTION=1` and `OLLAMA_KV_CACHE_TYPE=q8_0`. Polls until the API responds.
4. `pull_model` — Downloads `qwen3:8b` and `nomic-embed-text` if not already present.
5. `start_api_sidecar` — Launches the PyInstaller backend (which includes `eeg-worker/paradox-eeg-worker.exe` when built with `build_release.ps1`) and waits for health. The health probe confirms `eeg_subprocess.bundled_worker: true`.

If the backend goes offline during normal use (2+ consecutive health check failures), the frontend redirects back to `/setup` to re-establish the stack.

Licensing System

Because "open source" and "free for everyone forever" are not the same thing, Paradox includes an offline licensing system that controls who can run the software, which features they can access, and on which machines. It works without internet, without phone-home checks, and without the kind of DRM that makes people want to throw their computer out of a window.

Cryptographic Design

Licenses are **RSA-signed JSON payloads**. The workflow:

1. You (the vendor) generate an RSA-2048 keypair. The **private key stays secret**.
2. For each customer, you create a JSON payload (organization, tier, features, expiration, optional machine ID) and sign it with the private key.
3. The signed license is: `BASE64(payload).BASE64(signature)`
4. The customer pastes this into the app. The app verifies using the **embedded public key**.

$$\text{license_key} = \text{B64}(\text{JSON}) \cdot \text{B64}(\text{RSA_SIGN}_{\text{privkey}}(\text{JSON}))$$

Forgery requires the private key. Tampering is detected by signature mismatch. No internet needed.

License Payload Structure

```
{
  "organization_name": "Blue Ridge Neuro Clinic",
  "license_id": "uuid-here",
  "license_type": "pro",
  "max_users": 10,
  "max_locations": 1,
  "features": ["web_search", "deep_research", "python_analysis", "reflection"],
  "issued_at": "2026-03-26",
  "expires_at": "2027-03-26",
  "machine_id": "sha256-hash-or-empty"
}
```

Machine Binding

Optional but powerful. The machine fingerprint is a SHA-256 hash of:

```
raw = f"{uuid.getnode()}-{platform.system()}-{platform.machine()}-{platform.node()}"
machine_id = hashlib.sha256(raw.encode()).hexdigest()
```

If the license includes a `machine_id`, it is compared against the current machine's fingerprint at startup. Mismatch = license rejected. This prevents license sharing between machines without making the system call home.

Feature Flags & Tiers

Tier	Features Included
Community	web_search, fetch_url, query_memory, get_system_stats, get_datetime, boost_memory, ingest_document
Pro	Community + deep_research, python_analysis, reflection
Clinic	Pro + eeg, shell_command, discord
Enterprise	All features, multi-location support

Each tool in the registry has an optional `required_feature` attribute. When a licensed tool is called, the registry checks `license_manager.has_feature()` before execution. Unlicensed tools return an error message rather than silently failing.

Vendor Tools

- **CLI:** `python scripts/license_gen.py` — keygen, sign, verify, machine-id commands.
- **GUI:** `python tools/license_manager_gui.py` — standalone Tkinter application with tabs for Generate, Registry, Verify, Machine Info, and Keypair management. Dark theme. Persistent registry in `keys/license_registry.json`.

File Management

Research assistants that can't accept files are like librarians who refuse to handle books — technically present, but not doing the job they were hired for. Paradox has a file management system with smart routing, because if it is a good idea, you should go ahead and do it rather than waiting for someone to type a path into a terminal:

File Type	Extensions	Destination	Auto-Action
EEG recordings	.edf, .bdf, .set, .fif, .vhdr, .cnt	`eeg_workspace/`	Available for MNE analysis pipelines
Documents	.pdf, .txt, .md, .csv, .json, .yaml, .docx	`ingest_queue/`	Immediately ingested into ChromaDB memory

General	Everything else	`vault/uploads/`	Stored for manual reference

API Endpoints

- `POST /files/upload` — Multipart file upload (max 500 MB). Routes by extension, auto-ingests EEG and document files.
- `GET /files?category=all|eeg|document|general` — Lists files across managed directories.
- `DELETE /files/{category}/{filename}` — Removes a file.

Frontend Integration

The sidebar includes a `FilePanel` component with drag-and-drop upload, progress indicators, and a categorized file list with delete buttons. EEG files uploaded here are immediately available for the `run_eeg_pipeline` and `run_python_analysis` tools.

Use Cases

▼ Clinical QEEG Review

"Load the EDF from today's session, run the full pipeline, and tell me which channels show excessive theta."

The agent calls `run_eeg_pipeline`, which generates and executes the 24-step script. The pipeline loads the EDF, preprocesses it, runs ICA with ICLabel for artifact removal, computes band power topomaps, and produces a quality report. The agent reads the report and the band power metrics, then answers: "Channels Fz and Cz show elevated theta (4-8 Hz) power at 2.3× and 1.8× the mean, respectively. Here's the topomap: [link to output file]."

▼ Neurofeedback Protocol Research

"Find recent papers on alpha-theta neurofeedback for PTSD."

The agent calls `deep_research`, which queries PubMed, Google Scholar, and general web simultaneously. Results are fetched, cached, and synthesized. The agent returns a

Markdown summary with paper titles, key findings, sample sizes, and direct links to the sources.

▼ Custom EEG Analysis Script

"Load patient_003.edf, compute coherence between F3 and F4 in the alpha band, and plot it over time."

The agent writes a MNE-Python script using `run_python_analysis` that loads the file, filters to 8-13 Hz, computes spectral connectivity between the specified channel pair, and saves a time-frequency coherence plot. If the script fails (wrong channel name, missing montage), the agent reads the error, fixes the code, and retries automatically.

▼ Event Research

"When and where is the next Suisun Summit?"

The agent calls `web_search`. The multi-phase pipeline generates query variants (including concatenated keyword recovery for typos like "sui sun" → "suisun"), searches with staggered execution, probes for current-year URLs, and follows outbound links from discovered pages. Returns the event name, location, dates, and a clickable link to the official page — typically in under 50 seconds.

▼ Knowledge Building

Drop a PDF of a research paper into `data/ingest_queue/`. The file watcher detects it within 60 seconds, chunks the text (1200 chars, 150 overlap), embeds each chunk, and stores them in the library collection. In future conversations, the agent can recall specific findings from the paper via `query_memory`.

▼ System Monitoring

"How much VRAM is Ollama using right now?"

Routed via dual-model as AGENT (needs tool). The agent calls `get_system_stats`, which parses nvidia-smi CSV output. Returns: model name, VRAM used/total, temperature,

utilization percentage — all from a single tool call.

Paradox Solutions LLM v0.3.1 — A system built on the principle that if you're going to run an AI locally, you might as well build it properly. Now with a **bundled EEG worker** (MNE, SciPy, Plotly, scikit-learn, networkx, statsmodels — no separate Python science install required), **Coben-style Granger brain overlays**, **LORETA source localization**, **Mahalanobis 3D phenotype surfaces**, **interactive Trace Viewer**, proper **504 timeout handling** on the agent route, **version tracking** via a single `VERSION` file, and a two-script build pipeline that produces MSI/NSIS installers. Plus everything from v0.3.0: native desktop packaging, offline licensing, GPU optimizations, file management, REST EEG jobs with interactive dashboards, and agent tools for full pipelines and custom MNE scripts. Every layer documented. Every formula shown. Every decision explained — because black boxes are for magicians, and this is engineering.